Canonical source: 00-CHARTER/GUIDES/PROOF_AND_BENCHMARKS.md

Æ Orange AI Computer Proof And Benchmarks

Purpose

Orange is built around falsifiable operational claims. This guide defines the evidence vocabulary, current accepted results, reproduction surface, and the scope of every number.

The public principle is simple:

> Every surprising claim should tell the reader what ran, on what data, through which path, with which acceptance rule, and where the evidence lives.

Evidence Classes

Class	Establishes
Source	the implementation exists
Configuration	a route or role is declared
Unit test	a bounded contract behaves under the test inputs
Integration test	multiple real modules cooperate under the test inputs
Live probe	a deployed endpoint returned semantic state at a recorded time
Execution receipt	a named action ran and produced named evidence
Artifact proof	an output was independently parsed, decoded, or inspected
Held-out benchmark	behavior generalized across a fixed unseen case set
Ablation	the complete method beat a simpler component or baseline
Native UI proof	the packaged application displayed the expected runtime result

No evidence class silently expands into another. A model installed on disk does not prove it was routed. An HTTP response does not prove semantic health. A file's existence does not prove its content. A screenshot does not prove the backend unless both share a run or receipt identity.

Current Accepted Evidence Set

Blue Bench

Receipt:

`10-RECEIPTS/orange5-build/2026-08-28T03-40-44-768Z-blue-bench.json`

Result: 10 accepted lanes of 10 in that run:

1. Context Crystal and AtomSmasher.
2. AE Memory.
3. OrangeBrain routing.
4. Hermes.
5. Current Awareness.
6. AE Eyes.
7. No-ghost proof.
8. Atomic Orange conversation.
9. Party Line.
10. Fixer.

The run makes no external competitor-parity claim. It is an internal operational suite with exact-path freshness and evidence checks.

Integrated Operational Proof

Receipt:

`10-RECEIPTS/orange5-build/2026-08-28T03-42-45-242Z-integrated-operational-proof.json`

Accepted observations include:

- semantic runtime status `OPERATIONAL`;
- live OrangeLLM gateway;
- distributed compute-fabric selection;
- authenticated and executable Codexa rail;
- current Navigator route identity;
- Context Crystal held-out parity;
- memory quality and contradiction-debt resolution;
- Brain MCP live HTTP and dual-transport evidence;
- Hermes governed read and process actions;

- Captain Planet technical artifact validity.

Live Hermes Brain MCP Delegation

Receipt:

`10-RECEIPTS/orange5-build/2026-08-28T04-13-22-203Z-brain-mcp-delegation-live-proof.json`

Observed path:

```
Brain MCP request
-> parent governed filesystem.read
-> parent execution receipt
-> Hermes authorization gates
-> child report and child receipt
-> synthesis report and synthesis receipt
-> lease revocation
```

All ten named checks passed. Elapsed time was 11,409.53 ms. The result hash, receipt hashes, and receipt sequence numbers are preserved in the proof.

Context Crystal And AtomSmasher

Question

Can Orange construct a much smaller task workbench while preserving the held-out answer and exact source pointers?

Corpus

- 794 sources.
- 7,056,795 bytes.
- Five held-out cases.

Acceptance

- every case returns the required answer material;
- every required source pointer verifies;
- every case meets the declared operational-ratio threshold;
- the receipt hash validates.

Result

- 5/5 cases passed.

- Minimum operational context ratio: 1,422.901x.
- Maximum operational context ratio: 1,445.487x.

Definition

The operational context ratio is:

```
estimated source context required by replay
--------------------------------------------
tokens in the source-backed active workbench
```

It is workload-specific. The full cold source remains addressable. The number therefore describes workbench reduction under the benchmark contract, not a universal byte codec and not a guarantee for arbitrary prompts.

Reproduce

```
cd C:\AtomEons\Orange5
bun run bench:context-crystal-quality
bun run test:atomsmasher
```

Review the emitted case records, source hashes, ratios, and receipt hash rather than quoting the terminal summary alone.

AE Memory

Question

Does the hybrid retriever recover the expected project evidence more reliably than lexical-only or dense-only retrieval?

Cases

Twenty-three held-out queries spanning exact identifiers, semantic paraphrases, project history, failure memory, and contradiction precedence.

Result

Method	Cases passed	MRR	p95 latency
Lexical only	20/23	0.8435	342 ms
Dense only	21/23	0.8478	440 ms

Method	Cases passed	MRR	p95 latency
Hybrid	23/23	0.9058	445 ms

Hybrid p50 was 281 ms and maximum observed latency was 573 ms. Three contradiction-debt cases were recorded and resolved using the evidence law: fresh receipts and live probes outrank older prose.

Reproduce

```
cd C:\AtomEons\Orange5
bun run bench:memory-quality
```

The benchmark must pass retrieval quality and latency together. A faster method that misses the expected evidence does not win.

Brain MCP

Orange exposes two MCP transports:

- stdio for desktop and IDE clients;
- loopback Streamable HTTP for apps and model-to-model calls.

The integrated proof references a dual-transport receipt and verifies that the Codexa rail is reachable, authenticated, and executable. Transport-specific tool counts may differ because a desktop-safe stdio surface and an authenticated loopback surface serve different callers.

Reproduce

```
cd C:\AtomEons\Orange5
bun run test:orange:mcp
bun run proof:orange:mcp
```

Hermes

Hermes proof has two layers:

1. direct governed action proof for harmless filesystem and process actions;
2. Brain MCP delegation proof for mediated parent/child/synthesis work.

Each proof checks authorization and actual completion. Lease revocation is an acceptance condition, not cleanup commentary.

Reproduce

```
cd C:\AtomEons\Orange5
bun run test:orange:effector
bun run proof:orange:hermes-live
bun 03-BACKEND/brain-mcp-delegation-live-proof.mjs
```

Atomic Orange

Atomic Orange proof must use the native Tauri executable, not a Vite browser tab. A complete native proof records:

- application process identity;
- model/gateway request identity;
- time to first visible assistant content;
- final response completion;
- receipt identity shown in the app;
- BuildRun identity and chain validity;
- screenshot hash;
- native process and runtime evidence.

The current accepted Blue Bench contains a native conversation proof. A newer streaming/BuildRun build is being validated as a separate promotion and should replace the earlier evidence only after its native proof closes.

AE Eyes

Visual proof is strongest when one identity links:

```
source image hash
-> visual route
-> structured result
-> report
-> receipt
-> operator-visible surface
```

Backend health, an embedding worker, and a screenshot are useful evidence, but the complete visual crossing is the target contract. Remote URL inputs also require a negative SSRF/path-boundary test.

Captain Planet Creative Roles

Current accepted artifacts are technically valid: image/video files were independently decoded, video motion was measured, and speech/music WAV files were confirmed non-silent with stable hashes.

Technical validity and studio quality are separate evaluations. Studio promotion adds task-specific aesthetic rubrics, reference comparisons, human review, and artifact reproducibility.

Fixer

Fixer proof injects a controlled service fault, observes it, repairs it, checks neighboring services, and records recovery time. The live proof establishes the named recovery path. Broader reliability is expanded by feeding Fixer real failures from tests, runtime contradictions, route mismatches, stale receipts, and benchmark regressions.

Clean-Source Publication Rule

An internal benchmark may measure a dirty development tree. A publishable release claim requires an additional clean-source run:

1. record exact commit;
2. preserve the active worktree;
3. reproduce from a clean checkout;
4. run the named suite;
5. verify generated artifacts and hashes;
6. link CI or an equivalent independent run for that exact commit;
7. publish source, environment, receipt, and claim boundaries together.

This prevents a valid local result from being attributed to source that does not yet contain the implementation.

Citation Template

Use this form in a paper, post, or release note:

```
On the Æ Orange AI Computer held-out Context Crystal suite (794 sources, 7,056,795 bytes,
five cases), the source-backed active workbench preserved every required answer
and source-pointer check, with a minimum operational context ratio of
1,422.901x. This is a workload-specific operational-context result, not a
universal lossless-compression claim. Receipt: <path/hash>.
```

Independent Review Checklist

- Does the cited file exist?
- Does its SHA-256 match the manifest or receipt?
- Does the receipt name the exact implementation and input?
- Are denominator and units defined?
- Are quality checks present beside speed/compression metrics?
- Is the model route observed rather than inferred from inventory?
- Is native UI proof truly native?
- Is the result reproducible from the cited commit?
- Is the strongest nearby non-claim stated?

That checklist is not defensive language. It is the mechanism that makes an ambitious result durable.